

Function

method print(), input(), int(), float()
sum(), max(), min(), range()
 * function has !!
 * function has 1 or more

Syntax how to create function

def function_name (optional parameters, ...):

statement
 statement
 statement

function - block = code block
 which is function

↑
block

Syntax how to call function

function_name (optional value)

Simple function : basic

Ex! ↳ basic function is a block of code "Not" more

def print_myname():

Centre

Usage

E x 2

Create

Usage

1 Parameter function ($f(x)$)

* Parameter of the function

↑ parameter

Ex1 function ที่รับค่าเข้า แล้วให้ผลลัพธ์ออกมา
parameter

Chalk

Usage

✓ a = "Alice"

✓ greet_person(a) → "Alice" → name = "Alice" → Welcome Alice

* parameter do công việc thay đổi (input vào function)

Multiple parameters

↳ comma để viết parameter

EX1 - function có 2 tham số
- function có 2 input để trả về 2 kết quả

2 parameter

hai tham số

✓ def print_fullname(first, last):

print(first, last)

print(last, first)

first = "Nut"

last = "Yong"

Nut	Yong
Yong	Nut

✓ print_fullname("Nut", "Yong")

* 2 parameters

✓ first = "AAA"

✓ last = "BBB"

✓ print_fullname(last, first)

first = "BBB"

last = "AAA"

"BBB"

"AAA"

BBB	AAA
AAA	BBB

EX2 - function có 2 tham số để print ra kết quả

↳ function có 2 tham số 1) x, 2) y

def print_mul(x, y):

print(x * y)

2 * 2

4

```

print(x*y) → 2*2 → 4
print_mul(2, 2)
print_mul(3+2, 1+3*2) → 5*7 → 35

```

EX3 ถ้า ถ้าเฉลี่ย ของ คะแนน วิชา ของ บัณฑิต แล้วหาผลเฉลย
 วิชา math, sci, eng

```

def print_mean(math=20, sci=30, eng=40):
    avg = (math + sci + eng) / 3
    print(avg)

print_mean(20, 30, 40)

```

* positional argument

Keyword Arguments

↳ คือ การที่เรา ใส่ค่าให้กับ function ด้วย การ
 ใส่ค่า parameter ระบุชื่อค่าที่เราใส่ให้ function
 แทนที่จะใส่แค่ parameter

Usage print_mean (math=20,
 sci=30,
 eng=40)

* ลำดับไม่สำคัญ!

```

print_mean ( eng=40,
              math=20 )

```

variable


```
print_mern ( eng=40,  
             math=20  
             sci=30 )
```

* Note: if no positional or keyword
positional or keyword !!

```
print_mern ( 20, 30, end=40 ) ✓  
print_mern ( eng=40, 20, 30 ) ✗
```

Default Values

· range(s) → start=0, step=1

↑
default was range function

* function can have default parameter
in function and then function has parameter
and default value

Syntax: def function_name (param="default value"):
 statement
 statement

↑
if = default
default

Ex1: def greet (name="Not")
 print ("Hello", name)

1. Alice → Hello Alice

greet() ~~no value if default~~ → Hello Not

greet("Alice") → Hello Alice

* default argument is default

Ex 2: def print_mean(math, sci=25, eng=25):

avg = (math + sci + eng) / 3

print(avg)

print_mean(25) → math=25, sci=25, eng=25

print_mean(50, 50) → math=50, sci=50, eng=25

print_mean(30, 30, 30) → math=30, sci=30, eng=30

* Note: เราไม่สามารถใช้ค่าของ local variable
ใน function ภายนอก function

เช่น เราไม่สามารถใช้ค่าของ avg ภายนอก function

เพราะค่าของ local variable
จะหายไปเมื่อ function เสร็จสิ้น

print(avg) # non function ← Error!

return statement

* return คือ คำสั่ง นำค่า ส่งกลับ จาก function
คือค่าที่ function ส่งกลับให้

* return 10% 20% 30% 40% 50%

↳ break : คือ function ของ python ที่ใช้
(คล้าย break)

Ex 1

```
def give_s():  
    return 5
```

* ตัวเลข อาจมีทั้ง
เลขหลักสิบ เลขยี่
(เช่น $input(1)$)

✓ ~~x = give_s()~~ ⁵ ~~x = s~~
print(~~x~~) ⁵

15

Ex 2

```
def add(x, y):  
    return x + y
```

`result = add(2, 3)`
`print(result)`

$\Rightarrow X=2, Y=3 \rightarrow \underline{\text{return 5}}$

[5]

15

* γ_2 ଲେଉଟାଣି

$\text{add}(3, 5)$ #เท่ากับ $3 + 5$ แล้วค่อยไป
ใส่ใน arr

$$Z = \text{add}(\underbrace{\text{add}(1, 2)}_{1+2=3}, \underbrace{\text{add}(3, 4)}_{3+4=7})$$
$$1+2=3$$
$$3 + 4 = 7$$
$$\text{add}(3, 7) = 3 + 7 = 10$$

```
print( add(result, z) )
```

$$\underline{\underline{E \times 3}}$$

def add(a, b):

```
return a+b
```

```
def mul(x, y):
```

return $x * y = 6$

 $\cap$ r

1, 2, 3

```

return x * y = 6
def squared(x):
    return x ** 2
result = add(mul(2, 3), squared(3))
print(result) → 15

```

add(6, 9) → 15

EX4

function สำหรับ หา y
 $y = mx + c$
 - function รับ 3 ค่า : m, x, c
 - function ส่งกลับค่า y

```

def linear(m, x, c):
    y = m * x + c
    return y
z = linear(2, 2, 1)
print(z) → 5

```

*2 * 2 + 1 = 5*

EX5

function สำหรับ หา ค่าเฉลี่ย จาก list
 - รับค่า 1 ค่า คือ list ของตัวเลข
 - ส่งกลับค่า ค่าเฉลี่ย

```

def mean(numbers):
    return sum(numbers) / len(numbers)

```

*n = 6 คือ list ของตัวเลข
 [1, 3, 3, 4]*

$$\text{return } \underbrace{\text{sum}(\text{numbers})}_{10} / \underbrace{\text{len}(\text{numbers})}_{4} = 2.5$$

$$\text{print}(\text{mean}([1, 2, 3, 4])) \rightarrow 2.5$$

Ex 6

function ที่หาผลรวมของกำลังสองของตัวเลข

$$y = \sum_{i=1}^n x_i^2$$

- รับ list ของตัวเลข
- ส่งกลับ 1 ตัว

e.g. [1, 2, 3]

def sum_squared(numbers):

total = 0

for num in numbers:

total += num**2

return total

lst1 = [1, 2, 3, 4]

lst2 = [5, 6, 7, 8]

ss1 = sum_squared(lst1)

ss2 = sum_squared(lst2)

Multiple Return

* ส่งกลับ y หรือหลายค่า ด้วย comma

↓
ส่งกลับ

def mul_return():

return 3, 5, 7

return 3 ✗
return = ✗

return 3, 5, 7

~~return 3~~
~~return 5~~
return 7

(*) Usage : ถ้าฟังก์ชันมีค่ากลับ

Tuple แทนที่จะเป็น list ~~(3, 5, 7)~~ (3, 5, 7)

X = mul_return()

print(X) → (3, 5, 7)

print(X[0]) → 3

(*) Tuple ใช้ list สามารถ นำค่า
เก็บไว้ และ นำ ค่า มาใช้

X, Y, Z = [3, 5, 7]

a=3, b=5, c=7 = mul_return()

print(b) → 5

Summary

* parameters คือ ค่า ป้อน
~ input

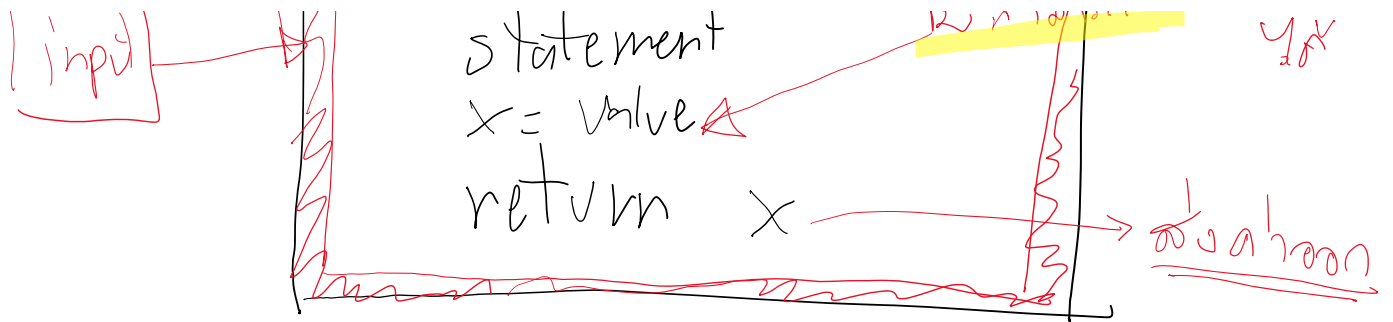
def function_name(parameters)

state ment

state ment

เขียน ชื่อ ฟังก์ชัน
ไว้

input



Usage: $a = \text{function_name}(\text{value1}, \text{value2})$